

Implementation Plan (16 GB VRAM · 10-Day Feasible Edition)

AIP-Speech — Acoustic Interference Profiling for Speech-LLMs

Single 16 GB GPU · < 3 GB storage · stimuli mixed on-the-fly yet fully inspectable and reproducible · companion to the proposal.

0. Principles and layout

- Manifest is metadata only:** (`speech_id`, `background_id`, `snr`, `condition`, `seed`). Stimuli are mixed in RAM at inference and discarded — **but any clip is regenerable bit-for-bit on demand** (`materialize`), a fixed inspection sample is persisted, and a per-stimulus diagnostics CSV records what happened.
- Generation ≠ scoring:** models write raw JSONL once; metrics are pure functions of it.
- All backgrounds are named public recordings** (ESC-50, DEMAND, MUSAN, NOISEX) — no synthesized babble. The only construction is SNR mixing per the released manifest.

```
aip-speech/
├─ data/
│  ├── bg/                # ~20 curated real backgrounds (ESC-50 + DEMAND + MUSAN + NOISEX)    (~1.8 GB)
│  ├── bg_scrambled/      # 8 phase-scrambled twins of the speech-like backgrounds          (~50 MB)
│  ├── speech_asr/        # LibriSpeech subset + Common Voice (streamed, accent-labelled)    (~350 MB)
│  ├── speech_kws/        # Google Speech Commands v2 subset (+ held-out injection-probe IDs) (~150 MB)
│  └── speech_saa/        # Speech Accent Archive subset (controlled fairness)              (~200 MB)
├─ descriptors/battery.parquet
├─ itembanks/{asr,kws,saa}.jsonl    # clean items + ground truth + demographic labels
├─ prereg/prereg.json
├─ manifests/{asr,kws}.csv          # metadata-only rows (no stimulus files)
├─ mixing/mix.py                   # on-the-fly mixer + materialize() + diagnostics
├─ inference/{model}/{task}.jsonl
├─ scoring/{wer,kws,injection,fairness,profile}.py
├─ results/
└─ checks/
   ├── inspection/              # persisted ~60s-clip sample for listening + checks
   ├── mix_diagnostics.csv       # per-stimulus: achieved SNR, LUF5, clip flag, bg-presence, mix-WER
   └── bg_presence.csv wer_constancy.csv scramble_validity.csv determinism.json
```

Total persisted ≈ 2.5 GB. Source corpora are real and citable; the stimulus *set* never lands on disk.

Stage 1 — Sources and item banks (Day 1)

Download **subsets**; stream the big one.

```
git clone https://github.com/karolpiczak/ESC-50 data/esc50          # ~600 MB diverse non-speech
# DEMAND: fetch 5 environments (real babble/speech-like + 1 stationary)
for env in PCAFETER PRESTO SPSQUARE OMEETING TCAR; do
  wget -q "https://zenodo.org/record/1227121/files/${env}.zip" -P data/demand && unzip -q data/demand/${env}.zip -d data/bg_raw
done
# NOISEX-92 babble (canonical multi-talker babble), MUSAN speech/music subset — fetch a handful of files
# LibriSpeech test-clean subset (~350 MB); Speech Commands v2 subset; Speech Accent Archive subset
python - <<'PY'          # Common Voice: STREAM, do not download the corpus
from datasets import load_dataset
cv = load_dataset("mozilla-foundation/common_voice_17_0", "en", split="validation", streaming=True)
PY
```

Build foreground banks (clean + ground truth + demographic labels), loudness-normalized to a fixed speech reference:

```
itembanks/asr.jsonl : {id, wav, transcript, source, accent, gender, age}
itembanks/kws.jsonl : {id, wav, keyword, is_target, probe_id?}
itembanks/saa.jsonl : {id, wav, transcript(fixed paragraph), accent, gender}
```

Acceptance: banks parse; ASR items span ≥4 accent groups × both genders; persisted speech < 1 GB.

Stage 2 — Battery, descriptors, scrambles, pre-registration (Day 2)

2.1 Curate ~20 real backgrounds

From the corpora above: ~12 ESC-50 non-speech events; 5 DEMAND/NOISEX/MUSAN speech-like (PCAFETER, PRESTO, SPSQUARE, OMEETING, NOISEX-babble); 2–3 MUSAN music/hubhub; 1 stationary (TCAR / MUSAN-noise). Take 1 channel, trim/loop to a common length, high-pass DC, loudness-normalize each to a fixed **background reference**.

2.2 Injection-probe backgrounds (reproducible, no synthesis)

A fixed published list of held-out Speech Commands clip IDs that utter the target keyword/digit; these are mixed as backgrounds at 0 dB. The list of IDs ships in the released manifest → anyone regenerates the identical probes.

2.3 Scrambled twins (speech-like subset only)

```
import numpy as np
def phase_scramble(x):
    X = np.fft.rfft(x); mag = np.abs(X)
    ph = np.exp(1j*np.random.default_rng(0).uniform(0,2*np.pi,size=mag.shape))
    return np.fft.irfft(mag*ph, n=len(x)).astype(np.float32) # same spectrum+energy, no structure
```

Validate (Stage 4) that twins carry no recoverable words.

2.4 Descriptors → descriptors/battery.parquet

speech_likeness (VAD/speech-classifier on bg), linguistic_content (Whisper-on-bg weighted word count), mod_2to8Hz, spectral_overlap(300–3400 Hz), harmonicity (HNR), stationarity (1/spectral flux), onset_density, loudness (LUFS), audioset_group.

2.5 Freeze prereg/prereg.json

Battery IDs + source corpus IDs, descriptor defs, SNR {+10,+5,0}, scramble method + which backgrounds, accent grid, metric defs, significance rule. **Nothing changes after the first model run.**

Stage 3 — The mixer: on-the-fly and inspectable (Day 2)

```
# mixing/mix.py
import numpy as np, soundfile as sf, pyloudnorm as pyln, whisper
SR = 16000; meter = pyln.Meter(SR); _asr = None

def _norm(x, lufs): return pyln.normalize.loudness(x, meter.integrated_loudness(x), lufs)

def mix(speech, bg, snr_db, seed, speech_lufs=-23.0):
    rng = np.random.default_rng(seed)
    s = _norm(speech, speech_lufs)
    if bg is None: return s.astype(np.float32)
    if len(bg) < len(s): bg = np.tile(bg, int(np.ceil(len(s)/len(bg))))
    off = rng.integers(0, max(1, len(bg)-len(s))); b = bg[off:off+len(s)]
    ps, pb = np.mean(s**2)+1e-9, np.mean(b**2)+1e-9
    b = b * np.sqrt(ps/(pb*(10**(snr_db/10))))
    y = s + b; peak = np.max(np.abs(y))
    return (y/peak if peak>1 else y).astype(np.float32)

def materialize(row, out_wav): # regenerate ANY clip exactly, for inspection
    sp = sf.read(row.speech_path)[0]
    bg = None if row.condition=="clean" else sf.read(row.bg_path)[0]
    sf.write(out_wav, mix(sp, bg, row.snr, row.seed), SR)

def diagnostics(row, y, s): # log "what happened" without saving audio
    global _asr; _asr = _asr or whisper.load_model("base")
    achieved_snr = 10*np.log10((np.mean(s**2)+1e-9)/(np.mean((y-s)**2)+1e-9))
    return {"stimulus_id":row.id, "achieved_snr":round(float(achieved_snr),2),
            "lufs":round(meter.integrated_loudness(y),2),
            "clipped":bool(np.max(np.abs(y))>=0.999),
            "mix_wer": None} # filled lazily on the inspection sample only (Whisper is slow)
```

Three inspection mechanisms (the answer to "how do I see intermediate steps?"): (1) materialize(row) regenerates any clip bit-for-bit; (2) a fixed ~60-row inspection sample is materialized to checks/inspection/ once; (3) diagnostics() writes checks/mix_diagnostics.csv for every stimulus.

Stage 4 — Manipulation checks (Day 3, before inference)

Run on the persisted inspection sample + a random regenerated subset: 1. **Background presence** — event/scene classifier confirms the intended background at each SNR → bg_presence.csv. 2. **Intelligibility constancy** — Whisper WER on the mix does not vary wildly across backgrounds at fixed SNR → wer_constancy.csv. 3. **Scramble validity** — twins match the real spectrum within tolerance and yield ~zero linguistic content → scramble_validity.csv. 4. **Determinism floor (replaces jitter floor)** — 50 clean items run twice, greedy; confirm identical outputs; record residual as one number → determinism.json.

Gate: low background-presence or words-in-twins = fix curation/scrambling first.

Stage 5 — Inference (Days 4-7, staged) — 16 GB pattern

```
class SpeechLLM:
    def generate(self, wav_np, task_prompt, system_prompt=None, max_new_tokens=64) -> str: ...
```

Adapters: Qwen2.5-Omni-3B (Thinker only), Qwen2-Audio-7B (4-bit), Phi-4-multimodal (4-bit), Gemma 3n-E4B, Kimi-Audio-7B (4-bit). **One model resident at a time;** `del model;` `torch.cuda.empty_cache()` between models. Understanding-only configs; **VRAM dry-run per model** before the full sweep. Batch 1-2 for 7B; larger for KWS.

```
for row in manifest:
    # metadata only
    sp = load(itembank[row.speech_id])
    bg = None if row.condition=="clean" else load(battery[row.background_id])
    wav = mix(sp, bg, row.snr, row.seed)
    log_diag(diagnostics(row, wav, _norm(sp,-23.0))) # numeric trace, no audio saved
    out = model.generate(wav, PROMPT[row.task])
    write_jsonl(f"inference/{model}/{row.task}.jsonl", {**row._asdict(), "raw": out})
```

Batching order: E1 (ASR+KWS) across 5 models (Days 4-5) → **Day-6 gate** → E2 scrambled + E3 SAA + text-oracle + (stretch) E4/extended (Days 6-7). Greedy decoding except the determinism check.

Acceptance: output count == manifest rows per (model, task); parse-failure rate low.

Stage 6 — Scoring, by experiment (Day 8)

- E1 (ASR, T1.1):** `jiwer` WER/CER; sub/del/ins split; ΔWER vs clean per item. **(KWS, T1.2):** Accuracy, FAR on non-target backgrounds, Miss.
- E2 (T2.1):** ΔWER(real) – ΔWER(scrambled) @ 0 dB; **BIR** = inserted tokens matching the background's own transcript/label vocabulary. **(T2.2): TIR** = FAR on injection probes vs scrambled vs generic-noise.
- E3 (T3.1 ecological):** per-subgroup ΔWER from the accent-labelled ASR run; **Robustness Gap, DRI. (T3.2 controlled):** content-matched per-accent ΔWER on SAA; descriptor×subgroup interaction.
- E4 (stretch): RER** = effect_with_instruction / effect_without; compliance rate.
- Significance:** paired permutation tests across items (p<0.05, corrected); semantic claims require real>scrambled; energetic claims require SNR-monotonicity.

Stage 7 — Analysis (Days 8-9)

- C-PROFILE:** mixed-effects $\Delta \sim \text{SNR} + \text{descriptors} + (1|\text{item}) + (1|\text{model}) + (1|\text{background})$; standardized coefficients, bootstrap CIs, VIF; dissociate via NOISEX-babble vs MUSAN-stationary. → **Fig 1.**
- C-FAIR:** Robustness Gap/DRI; descriptor×subgroup interaction; clean-gap overlay. → **Fig 3.**
- C-INJECT:** error taxonomy + BIR/TIR; real – scrambled. → **Fig 2** (+ appendix taxonomy).

Stage 8 — Reporting (Days 9-10)

Main: Fig 1 (descriptor law), Fig 2 (real vs scrambled), Fig 3 (disparate robustness); Table 1 (model × task summary), Table 2 (regression). **Appendix:** SNR dose-response, error/injection taxonomy, steerability RER, battery descriptor-space map, architecture view, per-background leaderboard, manipulation-check tables, full subgroup table, prior-art differentiation. Write the 4-page draft from E1 + C-PROFILE + E2 scrambled + E3 ecological cut. **Day-6 gate:** no descriptor law and no disparity = pivot to the robustness-null framing.

Appendix — Pitfalls (16 GB / reproducibility specific)

- One model resident at a time;** 4-bit + understanding-only; VRAM dry-run per model; keep a 3B/edge swap ready if a 7B OOMs.
- Never persist the stimulus set;** rely on `materialize()` + the inspection sample + `mix_diagnostics.csv` for observability.
- Loudness before SNR;** log achieved SNR per stimulus to catch mixing bugs.
- Greedy decoding** for core metrics → determinism check suffices; significance via paired permutation tests.
- Keep the 8 scrambled twins** — the cheap control that turns "noise hurts ASR" into "words hurt ASR." Don't over-build (speech-like backgrounds only, 0 dB only).
- All backgrounds named + public;** ship source IDs + seeds; no synthesized babble.
- Bake demographics into the ASR bank** so E3 ecological is a post-hoc cut, not a second grid.
- Watch model quirks:** Kimi defaults to ASR (force the task); speech-out models need Whisper transcription; quantized models need a sanity item first.
- Pre-register Day 2;** the profile/disparity story needs frozen thresholds.